

FreeRTOS基础（三）：动态创建任务

原创  VIP免费 最新推荐文章于 2026-08-11 19:41:38 发布 · 4.3k 阅读 ·  43 ·  71

上一篇博客，我们讲解了FreeRTOS中，我们讲解了创建任务和删除任务的API函数，那么这一讲，我们从实战出发，规范我们在FreeRTOS下的掌握动态创建任务的编码风格，达到实战应用！

目录

- 一、任务函数
- 二、动态创建任务的基本步骤
 - 2.1 使能FreeRTOS的API函数
 - 2.2 定义动态创建任务函数的入口参数
 - 2.3 编写任务函数
 - 2.4 主函数进行调用
 - 2.5 补充
 - 2.6 任务执行顺序
- 四、动态创建任务的API函数解析（选学）
- 五、任务优先级
- 六、总结

- 1、实验目的：学会 `xTaskCreate()` 和 `vTaskDelete()` 的使用
- 2、实验设计：将设计四个任务：`start_task`、`task1`、`task2`、`task3`

四个任务的功能如下：

- `start_task` 用来创建其他的三个任务
- `task1` 实现LED0每500ms闪烁一次
- `task2` 实现LED1每500ms闪烁一次
- `task3` 判断按键KEY0是否按下，按下则删掉task1

CSDN @未来可期，

一、任务函数

不论是动态创建任务还是静态创建任务，我们FreeRTOS都是在任务之间切换执行，那么任务函数就是我们单独要实现的功能，根据功能的不同统分割为一个个独立的无限循环且无法返回的函数。我们把这种函数称之为任务。即：**任务函数是没有返回值，并且是死循环的！**任务的形式：如下

```
void task1(void *arg)
{
    //初始化代码
```

```

while(1) //无限循环且不能返回
{
    具体实现的功能
}
//延时函数
}

```

1、为什么FreeRTOS的任务函数没有返回值？（可以将任务理解为线程）

1. 持续运行的任务

FreeRTOS 任务设计为长期运行，不像普通函数那样有明确的结束点。在嵌入式系统中，任务（或者称为线程）通常负责特定的功能，这些任务一直运行。例如，处理传感器数据、管理通信协议或维护系统健康状态等。这些功能**需要持续监控和响应外部事件或内部条件，因此任务函数通常循环。**

2. 任务调度

FreeRTOS 是一个实时操作系统，负责在多个任务之间进行调度。任务函数进入死循环后，会周期性地调用 FreeRTOS 提供的 API 函数（如 `vTaskDelay` 或 `xQueueReceive`），这些 API 会将任务置于阻塞状态，直到特定条件满足（延时时间到或者信号量接收到）。**这种设计允许 RTOS 的任务切换，确保系统的实时性和多任务处理能力。**

3. 没有返回值

由于任务函数设计为长期运行，因此它们不需要返回值。**任务的结束通常不是通过函数返回来实现的，而是通过其他机制，如任务删除（`vTaskDelete`）。**任务函数的主要目的是在系统运行过程中持续执行特定操作，而不是像传统函数那样在执行完特定操作后返回。

4. 系统稳定性和资源管理

任务函数设计为死循环还**有助于系统的稳定性和资源管理**。在 RTOS 中，任务的生命周期由系统管理，任务函数一旦启动，便由调度器根据调度策略进行管理。死循环的设计简化了任务的生命周期管理，避免了频繁创建和销毁任务带来的资源开销和复杂性。

2、为什么FreeRTOS任务函数的主体是一个死循环？

1、实时性：

通过使用死循环，任务**可以及时检查事件状态并作出相应的处理，以满足实时性。**

2、持续性：

将任务放在一个循环中，可以持续执行。如果任务函数没有死循环，而是在任务完成后直接返回，那么任务将会自动退出。这可能导致任务释放资源，而无法再次调度执行

3、提高资源的利用率：

只要任务不退出，就不需要重新获取资源，提高效率。

二、动态创建任务的基本步骤

2.1 使能FreeRTOS的API函数

在使用FreeRTOS任务创建函数之前，我们需要在配置文件里（`FreeRTOSConfig.h`）将宏 `configSUPPORT_DYNAMIC_ALLOCATION` 配置为 1 支持动态创建。利用 Ctrl+F 搜索即可。

```

/** 内存分配相关 */
// #define configSUPPORT_STATIC_ALLOCATION 1 // 静态创建任务
#define configSUPPORT_DYNAMIC_ALLOCATION 1 // 支持动态内存申请

```

CSDN @未来可期，静

2.2 定义动态创建任务函数的入口参数

通过上一讲我们知道动态创建任务的API函数如下：

```

BaseType_t xTaskCreate
(
    TaskFunction_t 函数指针          pxTaskCode, /* 指向任务函数的指针，其实就是函数名 */
    const char * const 字符串        pcName,     /* 任务名字，最大长度 configMAX_TASK_NAME_LEN */
    const configSTACK_DEPTH_TYPE usStackDepth, /* 任务堆栈大小，注意字为单位：1字=4字节 */
    void * const 泛型指针void*       pvParameters, /* 传递给任务函数的参数，一般传NULL */
    UBaseType_t 无符号长整型         uxPriority, /* 任务优先级，范围：0 ~ configMAX_PRIORITIES - 1 */
    TaskHandle_t * const 任务控制块结构体指针 pxCreatedTask /* 任务句柄，其实就是指指向任务控制块的指针/结构体 */
)

```

返回值为长整型，要用相同类型的变量接收 以p开头的都是指针

CSDN @未来可期，静

其实，我们需要定义的入口参数就是这个API函数的参数，提前定义好，然后传入参数，他就会自动的为我们创建好对应的任务，并且处于一种就绪状态。我们可以看到：

1、任务函数指针：

其实就是函数名，我们知道函数名就是函数的入口地址，就是一个函数指针

2、任务名字：

其实也就是函数名对应的字符串，要用双引号括起来

3、任务堆栈大小：

动态创建任务，任务的任務控制块以及任务的栈空间所需的内存，均由 FreeRTOS 自动从 FreeRTOS 管理的堆中分配，但是我们需要定义好大小，使用宏：

```
#define START_TASK_STACK_SIZE 128 //定义任务堆栈大小为128字（1字等于4字节）
```

4、传递给任务的参数：

不需要传参，我们直接给NULL即可；

5、任务优先级：

我们使用的是硬件的方式，因此，它要在0-31之间，使用宏定义即可：

```
#define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
```

6、任务句柄：

这个参数是指向任务控制块的指针，任务控制块TCB其实就是描述任务属性的一个结构体，一次他就是一个结构体指针，我们后续对任务的操作，都是通过该任务句柄进行操作，因此，我们需要提前定义好，然后传入即可，使用宏即可：

```
TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
```

从上面我们可以知道：其实我们只需要提前利用宏定义好三个参数即可，其他的参数只要任务函数编写好，便可以确定。示例如下：

```

/*****START_TASK任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
void start_task(void* args);

```

注意：

1. 为了编码规范，我们使用的宏都是大写，虽然较长，但是通俗易懂；
2. 使用API函数进行任务创建，里面的参数需要进行强制转换，以免报错。
3. 为了任务执行的顺序是按照我们设定好的优先级执行的，我们可以在创建任务的任務中，使用临界段保护，那么在这个任务体中，可以屏蔽中断优先级在5-15之内）比如切换任务的PendSV，此时，我们创建任务的过程中，不会进行任务的调度，然后我们创建任务结束后，在打开保护，此时不会对所有中断进行屏蔽，也就是任务切换PendSV(中断)才会进行任务调度。如下代码所示，在创建任务开始之前和创建任务之后面详细讲解。
4. 动态创建任务函数，有返回值，我们可以在编程时，对返回值进行判断，由此可以知道任务是否创建成功！

```

#include "stm32f4xx.h" // Device header
#include "stdio.h"
#include "FreeRTOS.h"
#include "task.h"
#include "dynamic.h"

/*****START_TASK任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/

#define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
void start_task(void* args);

```

```

/*****TASK1任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define TASK1_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define TASK1_PRIIO 2 //定义任务优先级，0-31根据任务需求
TaskHandle_t task1_handler; //定义任务句柄（结构体指针）
void task1(void* args);

/*****TASK2任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define TASK2_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define TASK2_PRIIO 3 //定义任务优先级，0-31根据任务需求
TaskHandle_t task2_handler; //定义任务句柄（结构体指针）
void task2(void* args);

/*****TASK3任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
#define TASK3_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
#define TASK3_PRIIO 4 //定义任务优先级，0-31根据任务需求
TaskHandle_t task3_handler; //定义任务句柄（结构体指针）
void task3(void* args);

```

开始任务用来创建其他三个任务，只创建一次，不能是死循环,同时创建完3个任务后删除开始任务本身

```

void start_task(void* args)
{
    taskENTER_CRITICAL(); //进入临界区*/

    BaseType_t xReturn; //定义接收函数返回值的变量

    xTaskCreate( (TaskFunction_t) task1,
                (char *) "task1",
                ( configSTACK_DEPTH_TYPE) TASK1_STACK_SIZE,
                (void *) NULL,
                (UBaseType_t) TASK1_PRIIO ,
                (TaskHandle_t *) &task1_handler );

    //任务1创建结果的判断
    if( xReturn == pdPASS)
    {
        printf("LED_Task create SUCCESS\n");
    }
    else
    {
        printf("LED_Task create FALL\n");
    }

    xTaskCreate( (TaskFunction_t) task2,
                (char *) "task2",
                ( configSTACK_DEPTH_TYPE) TASK2_STACK_SIZE,
                (void *) NULL,
                (UBaseType_t) TASK2_PRIIO ,
                (TaskHandle_t *) &task2_handler );

    //任务2创建结果的判断
    if( xReturn == pdPASS)
    {
        printf("LED_Task create SUCCESS\n");
    }
    else
    {
        printf("LED_Task create FALL\n");
    }
}

```

```

xTaskCreate( (TaskFunction_t)      task3,
             (char *)              "task3",
             ( configSTACK_DEPTH_TYPE) TASK3_STACK_SIZE,
             (void *)              NULL,
             (UBaseType_t) TASK3_PRIIO ,
             (TaskHandle_t *)      &task3_handler );

//任务3创建结果的判断
if( xReturn == pdPASS)
{
    printf("LED_Task create SUCCESS\n");
}
else
{
    printf("LED_Task create FALL\n");
}

vTaskDelete(NULL);    //删除开始任务自身，传参NULL

taskEXIT_CRITICAL();  /*退出临界区*/

//临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
}

```

2.3 编写任务函数

对每个任务具体实现的功能进行函数的实现：**需要注意，任务函数没有返回值并且是死循环的！**

```

/*****其余三个任务的任务函数，无返回值且是死循环*****/

/**任务1: 实现LED0每500ms翻转一次*****/
void task1(void* args)
{
    while(1)
    {
        printf("任务1正在运行! \n");
        GPIO_ToggleBits(GPIOF,GPIO_Pin_9 );
        vTaskDelay(500);    //FreeRTOS自带的延时函数，会进行任务切换调度

    }
}

/**任务2: 实现LED1每500ms翻转一次*****/
void task2(void* args)
{
    while(1)
    {
        printf("任务2正在运行! \n");
        GPIO_ToggleBits(GPIOF,GPIO_Pin_10 );
        vTaskDelay(500);    //FreeRTOS自带的延时函数，会进行任务切换调度

    }
}

/**任务3: 判断按键KEY0，按下KEY0,任务1删除*****/
void task3(void* args)
{
    while(1)
    {
        printf("任务3正在运行! \n");
        if(GPIO_ReadInputDataBit(GPIOE,GPIO_Pin_4)==0) //表示按键按下
        {
            if(task1_handler!=NULL) //防止重复删除

```

```

        {
            printf("删除任务1! \n");
            vTaskDelete(task1_handler);    //删除任务1, 传任务1的句柄
            task1_handler=NULL;
        }
    }
    vTaskDelay(10);    //FreeRTOS自带的延时函数, 会进行任务切换调度
}

}

```

此外，我们再自定义一个入口函数，用来创建开始任务，然后将要创建的任务全部放在这个开始任务中，主函数只需调用这个入口函数，即可在任务中，创建其他的任务，这样做，规范代码，梳理代码逻辑，清晰易懂任务的运行顺序！如下所示：

```

//FreeRTOS入口例程函数，无参数，无返回值，用来创建开始任务
void freertos_demo(void)
{
    xTaskCreate( (TaskFunction_t)    start_task,
                (char *)            "start_task",
                ( configSTACK_DEPTH_TYPE)  START_TASK_STACK_SIZE,
                (void *)            NULL,
                (UBaseType_t)  START_TASK_PRIO ,
                (TaskHandle_t *)    &start_task_handler );

    vTaskStartScheduler();    //开启任务调度器
}

```

2.4 主函数进行调用

在完成上述的编写后，主函数内部只需要引入对应的头文件，然后在函数内部调用相应的函数对使用到的外设进行初始化，然后调用入口函数即我们设定的优先级进行任务的调度，如下所示：

```

#include "stm32f4xx.h"                // Device header
#include "stdio.h"
#include "myled.h"
#include "mykey.h"
#include "myusart.h"

#include "FreeRTOS.h"
#include "task.h"
#include "dynamic.h"    //可以用来单独存放任务函数的声明以及配置相关的宏定义，然后直接引入头文件使用

extern TaskHandle_t Start_Handle;
/*使用任务句柄可以对任务操作，如果没有添加上面的单独头文件存放，
那么使用其他文件的全局变量利用extern关键字引入即可。*/

int main(void)
{
    //1、外设初始化
    My_UsartInit();
    LED_Init();
    KEY_Init();

    //2、调用入口函数
    freertos_demo();

}

```

2.5 补充

为进行模块化的编程，我们可以将创建相应的头文件可以用来单独存放任务函数的声明以及任务配置相关的宏定义，然后在主函数直接引入头文件，这样工程结构清晰易懂！

2.6 任务执行顺序

编写完程序后，一定要进行验证，验证程序是否按照我们设定的顺序及进行执行，类似于操作系统的线程同步问题！

首先主函数调用入口函数，在入口函数内部创建开始任务函数，该开始任务进入就绪状态，**启用任务调度器，调度器启动后，FreeRTOS 将控制，开始调度任务。**此时CPU就会去执行开始任务，然后，在开始任务中创建三个任务，注意：由于使用了临界保护：taskENTER_CRITICAL() 它会进入临界区*/ 它会对5-15优先级的中断进行屏蔽，即不会发生作用，其中PendSV是用来任务切换的内核中断，它的优先级是13，因此，会被屏蔽。就是说，我在创建三个任务的过程中，不会进行其他任务的切换，保证我的开始任务创建其他的三个任务不会被打断！！创建完三个任务后，它们就就绪状态，然后，再删除这个开始任务（因为每个任务只需要创建一次，多次创建占用堆栈内存，造成栈溢出！）此时，我在关闭临界区保护，taskEXIT_CRITICAL(); /*退出临界区*/，也就是打开所有中断，此时PendSV中断就会被打开，按照任务的优先级进行抢占式调度，分别执行任务2、任务1，在三个任务执行的过程中，加入适当的延时，他就会进行任务的切换，去就绪列表寻找优先级最高的任务去运行！

四、动态创建任务的API函数解析（选学）



五、任务优先级

在 FreeRTOS 中，任务的优先级决定了任务在系统中的调度顺序和执行时机。设定任务优先级是 FreeRTOS 任务创建过程中一个重要的步骤。

1、优先级的范围

FreeRTOS 任务优先级的范围由 configMAX_PRIORITIES 宏定义。该宏在 FreeRTOSConfig.h 文件中定义。通常，优先级的范围是从 0 到 configMAX_PRIORITIES - 1，优先级数值越大，优先级越高。

2、注意事项

- 优先级的相对性：**任务的优先级是相对的，系统中最高优先级的任务将获得最多的 CPU 时间。如果多个任务具有相同的优先级，调度器会采用轮转或其他调度策略在它们之间切换。
- 优先级反转：**在某些情况下，低优先级的任务可能会持有高优先级任务所需的资源，导致优先级反转问题。FreeRTOS 提供了优先级继承机制解决这个问题。
- 优先级设定的策略：**设定优先级时，需要考虑任务的重要性和时间敏感性。**实时性要求高的任务应设定较高的优先级，而非实时任务可以设定较低的优先级。**
- 避免过高优先级：**设定任务优先级时要避免将所有任务都设定为过高的优先级，这样会导致系统缺乏灵活性，可能导致低优先级任务得不到执行。

六、总结

通过以上的介绍，是不是觉得相比裸机开发确实提升了不少的难度，这就是实时性带来的，万事有利必有弊，多看几遍，相信你对动态创建任务清晰的认识，其实步骤也是非常简单的，接下来去实践吧！熟练后就不难了，万事开头难！

温馨提示：

对于某个需要知道具体函数的实现的，我们可以双击函数然后直接跳转到定义处，或者Ctrl+F 搜索，也可以去官网查看对应的使用实例：
<https://www.freertos.org/>。

至此，动态创建任务就已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞关注的支持是我前进最大的动力！下期再见！

标签 #java #开发语言